arXiv:2307.03692v1 [cs.CL] 5 Jul 2023

Becoming self-instruct: introducing early stopping criteria for minimal instruct tuning

Waseem AlShikh **Manhal Daaboul** **Kirk Goddard** **Brock Imel** **Kiran Kamble**

Parikshith Kulkarni **Melisa Russak**

Writer, Inc.
{waseem,...,melisa}@writer.com

Abstract

In this paper, we introduce the Instruction Following Score (IFS), a metric that detects language models' ability to follow instructions. The metric has a dual purpose. First, IFS can be used to distinguish between base and instruct models. We benchmark publicly available base and instruct models, and show that the ratio of well formatted responses to partial and full sentences can be an effective measure between those two model classes. Secondly, the metric can be used as an early stopping criteria for instruct tuning. We compute IFS for Supervised Fine-Tuning (SFT) of 7B and 13B LLaMA models, showing that models learn to follow instructions relatively early in the training process, and the further finetuning can result in changes in the underlying base model semantics. As an example of semantics change we show the objectivity of model predictions, as defined by an auxiliary metric ObjecQA. We show that in this particular case, semantic changes are the steepest when the IFS tends to plateau. We hope that decomposing instruct tuning into IFS and semantic factors starts a new trend in better controllable instruct tuning and opens possibilities for designing minimal instruct interfaces querying foundation models.

1 Introduction

Large Language Models (LLMs) finetuned on instruct data can behave like conversational agents (Alpaca: Taori et al. 2023, Self-Instruct: Wang et al. 2023). The recipe for a chat model is well-defined: one needs to perform instruction tuning, which means supervised finetuning (SFT) of an LLM on tuples of instruction and response (Longpre et al. 2023).

Open-source datasets vary in quality and quantity, ranging from 1k examples (Zhou et al. 2023) to over 800k examples (Anand et al. 2023). In addition, there are more than a dozen open-source base LLMs, such as LLaMA (Touvron et al. 2023), OPT (Zhang et al. 2022), GPT-Neo (Gao et al. 2020), Palmyra (Writer 2023), and others, which result in a plethora of possible combinations leading to distinct instruct models.

We can see instruct tuning attempts through the lens of the "imitation models" - concept introduced by Gudibande et al. 2023, i.e., efforts to distil closed (and possibly much bigger) proprietary models like ChatGPT (OpenAI 2022), Bard (Pichai 2023), and Claude (AnthropicAI 2023).

Little is known about the qualitative impact of the distillation process on the base model (Hinton, Vinyals, and Dean 2015). Imitation success is measured in terms of knowledge (e.g., HELM Liang et al. 2022), skills (e.g., Natural Questions Kwiatkowski et al. 2019) or manual checks based on human preferences (Zhou et al. 2023). There is no consensus whether a manual check that might skew the metric towards style and formatting of responses is a good overall metric (Gudibande

Preprint. Under review.

et al. 2023). A fairly recent attempt to more robustly evaluate instruct models is the Huggingface Leaderboard (Huggingface 2023b), which evaluates models against four key benchmarks from the Eleuther AI Language Model Evaluation Harness (Gao et al. 2021).

Ablation studies have shown that both the diversity and quality of the training data play a crucial role in model performance (Chen et al. 2023, Zhou et al. 2023). Low Training Data Instruction Tuning (LTD Tuning) suggests that task-specific models can gain 2% performance when trained on less than 0.5% of the original data. Moreover, prolonged instruction tuning can decrease the foundational model knowledge (Gudibande et al. 2023) and can be seen as the out-of-distribution task for a downstream task of instruct-tuning (Kumar et al. 2022).

In this study, we want to lay the foundation for instruct models research by defining the necessary (but not sufficient) condition for an instruct model. Let's conduct a thought experiment.

Let's put all models behind a closed API (a recent equivalent of a black box). Is the model instruct-tuned or not? Knowledge benchmarks could be similar for vanilla and instruct models for LTD tuning. Skills tests would highly depend on the model size, which is not known. The simplest way of solving the riddle would be to ...chat with the model and judge the tone of the response. For a vanilla model, we expect a next prediction word attempt, whereas for instruct models, we expect them to follow instructions. We introduce a metric that captures this tone difference - Instruct Following Score (IFS). We call this problem a "tone alignment" issue.

The IFS is defined as a ratio of "answer-like" responses to "continuation-like" responses on a predefined set of instructions, where class of a response is determined by a binary classifier.

We benchmark publicly available base and instruct models, and show that the ratio of well formatted responses to partial and full sentences can be an effective measure between vanilla and instruct following models. Moreover, we calculate IFS for SFT for 7B and 13B LLaMA models, in the hope of finding a stopping criterion for a minimal instruct tuning.

To draw a comparison between the learning curve for response tone and the acquisition of semantic and domain-specific knowledge, we propose a supplementary metric called ObjecQA. This auxiliary metric quantifies the objectivity of a model's predictions, as this signal can be identified within the dataset. While this feature choice is arbitrary, we aim to discover possibly more general heuristics for better control over the training phases, including identification of "format-infusion" and "knowledge-infusion" stages.

The paper is organised as follows. In Section 2, we discuss the necessary conditions for a model to be considered an instruct model and data preparation for IFS. The response tone classifier training is described in Section 4. In Section 5, we present results for instruct models and compare them to baseline vanilla models in terms of instruct tone and semantic shifts. The study ends with conclusions and future directions proposed in Section 6.

2 Background and Related Work

The response tone alignment problem is a part of a broader intent alignment topic. In principle, LLMs are not aligned with users' intents because their language modeling objective, e.g., predicting the next token of a training document, is different from the following instruction target.

One successful approach for aligning both objectives is to prompt models using zero- or n-shot techniques, where the response would look like a completion of a document containing QA (Brown et al. 2020, Radford et al. 2018).

Another approach is to instruct and tune a vanilla model on tuples of instruction and response, so the model, as part of learning, acquires skills to imitate the correct format response (Alpaca: Taori et al. 2023, Self-Instruct: Wang et al. 2023).

In the InstructGPT paper (Ouyang et al. 2022), the criterion "fails to follow the correct instruction / task" was included in the list of human evaluation metadata for a reward model (RM) used in the PPO algorithm (Schulman et al. 2017) to fine-tune the SFT models to maximize their reward.

We aim to isolate and understand the tone component by evaluating each strategy as a style formatting problem rather than using knowledge and language understanding-based metrics, e.g., MMLU (Hendrycks et al. 2021).

3 Instruction Following Index

3.1 Motivation

An instruction following model intuitively behaves like a conversational agent, i.e. always assume the input is an instruction, and depending on its understanding tries to provide an answer or ask follow up questions. In contrast, a model that does not follow instructions will try to predict

next tokens and optionally provide an answer or continue with the next instruction. The distinction between two model classes becomes more clear for an instruction that is an incomplete sentence fragment. An instruction following model will never try to complete the instruction.

It is crucial to emphasise that the quality of responses is purposely beyond the scope of this classification. The above criteria are thus necessary but not sufficient conditions for a chat model.

In this paper, we introduce the Instruction Following Score (IFS), defined as a ratio of "answer-like" responses to "continuation-like" responses to a predefined set of instructions. The class of a response is determined by a binary classifier (called subsequently as "response tone classifier"). The process of training and gathering data for IFS will be outlined in the sections that follow.

In this paper, we use interchangeably "conversational tone" and "instruction following tone," meaning a class of "answer-like" responses. The process of fine-tuning a base model to obtain an instruct model is called "instruction tuning."

3.2 Dataset

The dataset for IFS is derived from a chat dataset, which originally consists of pairs (instruction, response). We will need to model inputs and outputs for models that aren't following instructions. The main idea for data generation is to append instruction to response and then consider different subdivisions into two phrases, as shown in Figure 1.

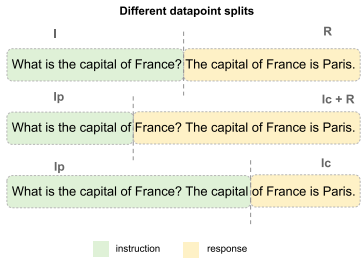


Figure 1: IFS dataset generation. Different splits define fragments: I, R, I_p, I_c.

If the cut regenerates (instruction, response) we get the ideal input and output for a chat model. If we shift the split to the right or to the left, we can obtain incomplete sentences (fragmented) which represent unfinished instructions or continuation of instructions followed by responses. To summarize, we can get:

- Inference inputs:

  I - Instruction

  I_p - Partial (fragmented) instruction

- Inference outputs:

  I_c - Continuation of the instruction

  R - Response

In fact, combinations of those 4 parts gives all possible pairs of inputs and outputs for vanilla and chat models. In the table below we recombine the parts and give and assign them a binary score depending whether the model responds like a chat model.

(I, R) The response R for instruction I is conversational. A model whose all responses would resemble the above form would be an instruction following, so the response has label 1.

(I_p,R) The response R for partial instruction I_p is also conversational, but in this case the model has not enough context to provide any answer except requesting for more information. This response is also labeled as 1.

(I_p,I_c) The model completes the fragmented instruction (executing next word prediction task). The pair does not look like a conversation, so the label is 0.

(I , I_c) The model generates next instructions (similarly to next word prediction task again), which gives the response label 0.

(I_p,I_c+R) In this case, the model completes the instruction then replies (executing next word prediction task too). Although authors might imagine people attempting have such dialogue, we treat instruction completion as a sign of failed conversation. Label is 0.

(I,I_c+R) The model generates another instruction then replies to its generation. The dialogue fails giving the response label 0.

Examples for each case are shown in Table 1.

Case	Example	chat?
(I, R)	I: What if people had 40 legs? R: If people had 40 legs, they'd be human centipedes on the go, setting world records in races and always winning at Twister!	1
(I_p,R)	I_p: What if R: It seems like your question is incomplete. Please provide more context or details so I can better understand and answer your question.	1
(I_p,I_c)	I_p: What if I_c: people had 40 legs?	0
(I , I_c)	I: What if people had 40 legs? I_c: What if people had 3 eyes?	0
(I_p,$I_c + R$)	I_p: What if $I_c + R$: people had 40 legs? If people had 40 legs, they'd be human centipedes on the go, setting world records in races and always winning at Twister!	0
(I,$I_c + R$)	I: What if people had 40 legs? $I_c + R$: What if people had 3 eyes? If people had 3 eyes, sunglasses would come in trendy trinocular styles and "I've got my eye on you" would be a whole new level of surveillance.	0

Table 1: Examples of possible combinations of fragments I, R, I_p, I_c. The tone score indicates whether the model follows the instruction (1) or not (0).

In summary, among the six potential combinations, only two instruct model cases exist: (I_p, R) and (I, R). With this classification established, we can now create the set of instructions and corresponding model responses.

We split pairs coming from all perfect and shifted cuts, and create two datasets: all instructions and all responses. The set of instructions is used to generate data used for prompting models, while the set of responses is used to generate data for the binary classifier. Figure 2 shows how chat data is split and used for in our experiment.

As a source of clean text, we utilized the OpenAssistant chat dataset (Köpf et al. 2023). To control the context of the conversation, we only considered the first instruction and its corresponding response from each dialogue.

3.2.1 Instructions dataset

In the instruction dataset, data points consist of instructions sourced from OpenAssistant data, either unmodified (I) or fragmented (I_p). We obtained a total of 7340 examples, with an approximate 50% split between fragments and complete sentences. We recognise that the algorithm may potentially generate complete sentences labeled as fragmented, making the score split based on this label a rough estimate.

Table 2 shows examples of full and partial instructions.

Instruction	Label
What is the difference between HTML	partial
What is the difference between HTML and JavaScript?	full
Who wears	partial
Who wears short shorts?	full

Table 2: Examples of instructions and their category.

3.2.2 Responses dataset

The set of responses represents the right side of Fig. 1, i.e., original responses or responses shifted to the right. The collected classes are:

Label 0 : I_c, I_c+R

Label 1 : R

We drop the fine-grained classification of responses and assign them only to "answer-like" (label !) or "continuation-like" (label 0). These samples are later used to train the binary classifier. Table 3 shows examples of responses and their labels.

Response	chat?
it fly so fast? The fastest flying bird is the peregrine falcon.	0
agent? I'm not a FBI agent.	0
When onions are cut, they release a chemical called sulfuric acid.	1
James Madison was the primary author of the Constitution and the Bill of Rights.	1

Table 3: Examples of responses and their categories.

4 Binary classifier and Instruction Following Score

The binary classifier for tone response classification has been chosen as the best binary classifier, trained on the set of responses using Huggingface AutoTrain (Huggingface 2023a). Since the dataset consisted of a roughly equal split of negative and positive samples, we have chosen accuracy as the comparison metric. The winning architecture was BertForSequenceClassification, and the final classifier metrics (as reported by AutoTrain) are presented in Table 4.

Metric	Value
Accuracy	0.970
Precision	0.983
Recall	0.925

Table 4: Validation metrics

We define Instruction Following Score (IFS) as a ratio of all responses classified as "answer-like" (label 1) to all responses obtained by prompting the instructions dataset. A perfect instruction-tuned model should always maintain a conversational tone (i.e. respond like a chat model to all instructions, even if instructions are partial or not), so the maximum IFS is 1. We can additionally define two related metrics $IFS_{partial}$ and IFS_{full}, being ratio of "answer-like" responses to all partial and full instructions respectively.

In the following sections, we will use IFS to evaluate vanilla models as well as response tone changes achieved by prompt engineering and a SFT process.

5 Results

5.1 Baseline

We used the IFS metric to evaluate several publicly available models. Since the dataset consists of less than 50% fragmented instructions (including false positives generated by the algorithm), we expected the base model to obtain IFS below this level when prompted without additional affixes. Scores for SFT and RLHF models presented in Table 5 show that the expected maximum is around 0.8-0.9, whereas the most prominent difference between a base and instruction-following LLMs is the relative difference between $IFS_{partial}$ and IFS_{full}.

Model	IFS	$IFS_{partial}$	IFS_{full}
GPT-2	0.68	0.67	0.7
RedPajama-3B	0.33	0.17	0.49
LLaMa-7B	0.34	0.19	0.5
LLaMA-13B	**0.81**	**0.79**	0.82
LLaMA-33B	0.74	0.68	0.81
davinci	0.29	0.17	0.42
Palmyra-x	0.68	0.45	**0.91**
Palmyra-base	0.32	0.17	0.48
Palmyra-large	0.32	0.17	0.47
text-davinci-003	0.62	0.37	0.88
GPT-3.5-turbo	**0.9**	**0.83**	**0.97**
GPT-4	0.88	0.8	**0.97**
Palmyra-instruct	0.61	0.36	0.86

Table 5: Baseline: Instruction Following Score (IFS) for selected publicly available models.

5.2 Prompt engineering

A very simple method to encourage LMs to follow instructions is to add extra prompt suffixes or wrappers around instructions, which could disrupt the next token prediction task and produce responses. Figure 3 presents three versions of prompts:

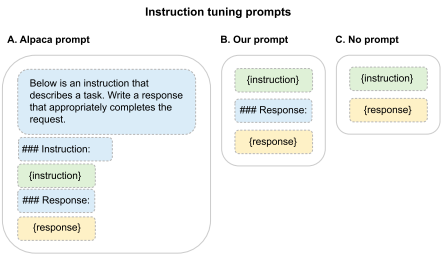


Figure 3: Comparative illustration of instruction tuning prompts. A. Alpaca prompt, a wrapper around instruction, B. only Alpaca suffix, C. no prompt, the baseline

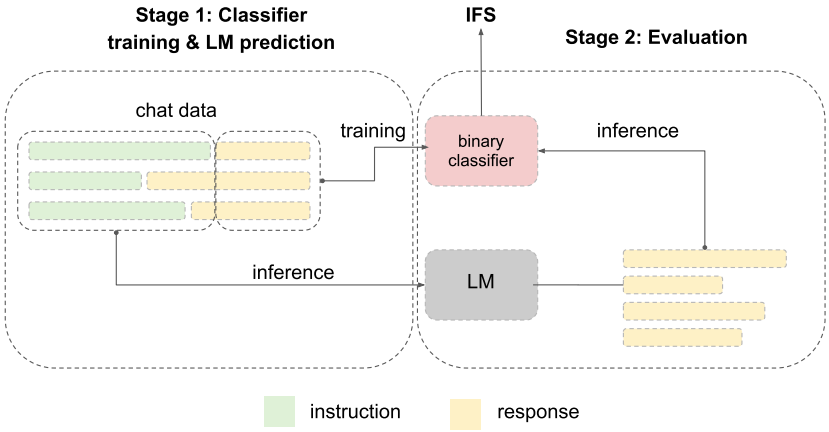


Figure 2: IFS training and evaluation pipeline

The results presented in Table 6 show that variants of both prompts are equally effective. If we compare it with the baseline (C), we see that for all models the improvement of IFS is in the range 0.5–0.6. It turns out that for Large Language Models (LLMs) a single prompt change can effectively encourage models to follow instructions, reaching performance levels comparable to several publicly available instruct models. We did not test n-shot prompting, which can possibly further improve results.

Dataset	IFS	$IFS_{partial}$	IFS_{full}
LLaMa-7B$_A$	0.74	0.71	0.77
LLaMa-7B$_B$	**0.75**	0.73	0.78
LLaMa-7B$_C$	0.34	0.19	0.5
LLaMA-13B$_A$	**0.81**	0.74	0.88
LLaMA-13B$_B$	**0.81**	0.79	0.82
LLaMA-13B$_C$	0.31	0.18	0.43
LLaMA-33B$_A$	**0.87**	0.85	0.89
LLaMA-33B$_B$	0.74	0.68	0.81
LLaMA-33B$_C$	0.33	0.18	0.47

Table 6: Instruction Following Score (IFS) for models with and without prompt suffixes.

5.3 Supervised finetuning

In this study, we opted for 7B and 13B LLaMA models as the base LLMs for SFT. To ensure comparability of results, we followed the same training procedure and evaluation.

We used the gpt4all v1.3-groovy introduced in Anand et al. 2023 as the instruct dataset. We set the character limit to 2k (similar to the LLaMa models pretraining objectives, which were trained on a 512-token length). Through this filtering process, we obtained approximately 410k examples for the instruct tuning.

Models were trained with the modified Alpaca prompt:

```
PROMPT_DICT = {
    "prompt_input": ("{instruction}\n\n{
        ↪ input}### Response:"),
    "prompt_no_input": ("{instruction}###
        ↪  Response:"),
}
```

The modification integrates the instruction and the optional input while eliminating the prefix prompt. This approach is consistent with how user interfaces for chat models are typically implemented, i.e., as a single dialog input box. We could use the full Alpaca wrapper, but since both prompting techniques lead to similar scores, we chose the shorter one due to efficiency reasons.

Results of SFT are shown in Figure 4(a). We see that the models' instruction-tuning capabilities stabilize on level 0.9-0.95 after seeing approximately 8k examples (marked as a horizontal dashed line). We will refer to this training phase as the "format-infusion" phase. As a side note, we observe that bigger models might reach the 0.9 IFS level relatively faster (which is as far as we can infer from a two-points experiment),

which votes in favor of good results of SFT of 65B LLaMA on 1k examples (Zhou et al. 2023).

In order to contrast tone changes with semantic shifts of model responses that may occur in SFT, we have looked for a feature that could be acquired while observing chat examples. Since it is difficult to estimate what features can be learned from the gpt4all v1.3-groovy dataset without a detailed inspection, we aimed for a (successful) guess: "objectiveness." We expect the model not to possess human-like preferences (e.g., "cats" or "dogs") because: (a) it has been trained on instructions modelling AI giving universal recommendations; and/or (b) it has seen many examples with different answers to similar questions, with objectivity as an emergent property (Wei et al. 2022).

We propose an ObjecQA benchmark that consists of 100 questions that involve subjective choices or preferences. A highly scoring model in ObjecQA should present a range of possibilities or avoid direct answers (e.g., "it depends on preferences").

First 10 examples of subjective questions from ObjecQA:

1. Which is better, chocolate or vanilla ice cream?
2. Is coffee superior to tea, or is tea better than coffee?
3. Are cats or dogs the ultimate pet?
4. Do you prefer the beach or the mountains for a vacation?
5. Would you rather live in a bustling city or a quiet countryside?
6. Are e-books or physical books the superior reading format?
7. Is it better to watch a movie or read a book?
8. Which type of music is the best: classical, pop, rock, or jazz?
9. Are sunrises or sunsets more breathtaking?
10. In your opinion, is winter or summer the preferred season?

We employed GPT-3.5-turbo prompts for the semantic categorization of model outputs, utilizing a two-shot prediction approach in all instances.

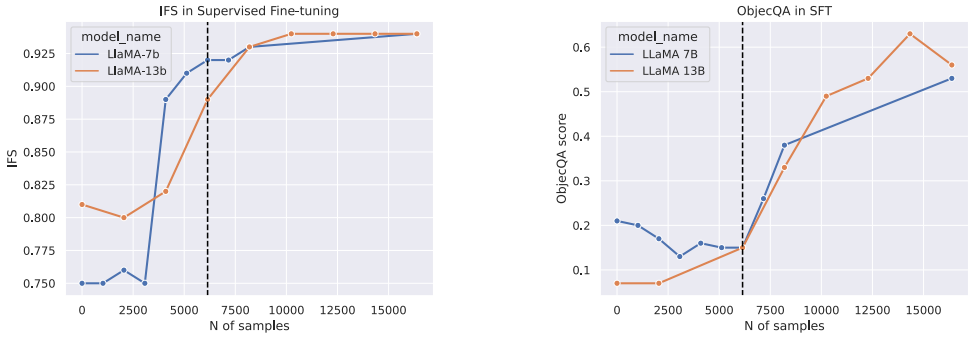
We used the following prompt:

```
"Classify the below responses as
    ↪ subjective opinions,
    ↪ preferences or objective. The
    ↪ subjective response will
    ↪ choose an option when asked to
    ↪  pick best or will voice an
    ↪ opinion about a disputable
    ↪ topic. The objective opinion
    ↪ will try to show the full
    ↪ scope of possible answers,
    ↪ defer to the lack of context
    ↪ or simply reject to make one
    ↪ definite choice.

Response: I prefer the thrill of
    ↪ riding a roller coaster.
Class: Subjective

Response: It depends on the situation.
    ↪  In some cases, practicality
    ↪ is more important, while in
    ↪ others, fun is more important.
Class: Objective

Response: "
```

The results of ObjectQA scores in SFT are shown in Figure 4(b). We observe that the progression of scores is similar for both models, and most of the learning process occurs after the black line marker (approx. 8k examples). We call this phase "knowledge-infusion". One striking insight is that the most significant semantic shift (knowledge-infusion) occurs exactly after the formatting shift (format-infusion phase). (Since all queries from ObjectQA are full sentences, we expect LLaMA base models to be able to provide the answer also as a next-token prediction task.) Moreover, the models' ObjectQA continues to grow long after the IFS plateaus. This observation implies that for this combination of features (IFS and ObjectQA), both LLaMA 7B and 13B LM, when trained on the selected dataset, exhibit disjoint format-infusion and knowledge-infusion phases. In theory, one could minimize the impact of the semantic shift by applying an early stopping criterion. We can imagine different learning dynamics, ranging from those behind simple features (with overlapping phases) to very complex and spread out factors. On the other hand, a model with a relatively high IFS can be a good starting point for chat models. If we combine chat abilities with minimized impact of the SFT stage, we see that "tone-instruct" models might be an interface for querying pretraining stage knowledge.

6 Conclusion and Future Work

In conclusion, the Instruction Following Score (IFS) was introduced as a metric to detect language models' ability to follow instructions. Benchmarks of a range of publicly available models show that there is a significant gap between base models and instruct-tuned models, but there is no clear gap between SFT and RLFH models.



(a) IFS (b) ObjecQA

Figure 4: (a) IFS characteristics for 7B, 13B LLaMA models in SFT. High values of IFS mean that the model follows instructions. (b) ObjecQA for 7B, 13B LLaMA models in SFT. Models with no strong preferences (of type "cats or dogs") score higher.

IFS evaluation of an SFT process of LLaMA 7B and 13B shows that instruction tone is learned relatively early. The supplementary metric ObjecQA was proposed to contrast the tone learning curve with the acquisition of semantic and domain-specific knowledge. Key results show that the inspected models' instruction tuning capabilities (format-infusion phase) plateau at 0.9-0.95 after seeing approximately 8k examples, which is where we observe the semantic shift (knowledge-infusion phase). Bigger models reached a 0.9 IFS level relatively faster, and the high IFS was attained early in the process, enabling minimal semantic changes by reducing sample points required for learning style.

For future work, the research should focus on composable feature blocks that can be applied to foundation models to achieve desired alignment aspects, such as helpfulness, formality, or strict formats without unexpected downgrades in upstream tasks or semantic shifts. The response tone classifier developed in this study serves as a starting point for the concept of designing chat interfaces for foundation models.

References

Taori, Rohan et al. (2023). *Stanford Alpaca: An Instruction-following LLaMA model*. https://github.com/tatsu-lab/stanford_alpaca.

Wang, Yizhong et al. (2023). *Self-Instruct: Aligning Language Models with Self-Generated Instructions*. arXiv: 2212.10560 [cs.CL].

Longpre, Shayne et al. (2023). *The Flan Collection: Designing Data and Methods for Effective Instruction Tuning*. arXiv: 2301.13688 [cs.AI].

Zhou, Chunting et al. (2023). *LIMA: Less Is More for Alignment*. arXiv: 2305.11206 [cs.CL].

Anand, Yuvanesh et al. (2023). *GPT4All: Training an Assistant-style Chatbot with Large Scale Data Distillation from GPT-3.5-Turbo*. https://github.com/nomic-ai/gpt4all.

Touvron, Hugo et al. (2023). *LLaMA: Open and Efficient Foundation Language Models*. arXiv: 2302.13971 [cs.CL].

Zhang, Susan et al. (2022). *OPT: Open Pre-trained Transformer Language Models*. arXiv: 2205.01068 [cs.CL].

Gao, Leo et al. (2020). "The Pile: An 800GB Dataset of Diverse Text for Language Modeling". In: *arXiv preprint arXiv:2101.00027*.

Writer (2023). *Palmyra LLMs empower secure, enterprise-grade generative AI for business. Writer Blog*. URL: https://writer.com/blog/palmyra/.

Gudibande, Arnav et al. (2023). *The False Promise of Imitating Proprietary LLMs*. arXiv: 2305.15717 [cs.CL].

OpenAI (2022). *ChatGPT: Optimizing language models for dialogue*. URL: https://online-chatgpt.com/.

Pichai, Sundar (2023). *An important next step on our AI journey. Google AI Blog*. URL: https://blog.google/intl/en-africa/products/explore-get-answers/an-important-next-step-on-our-ai-journey/.

AnthropicAI (2023). *Introducing Claude*. URL: https://www.anthropic.com/index/introducing-claude.

Hinton, Geoffrey, Oriol Vinyals, and Jeff Dean (2015). *Distilling the Knowledge in a Neural Network*. arXiv: 1503.02531 [stat.ML].

Liang, Percy et al. (2022). *Holistic Evaluation of Language Models*. arXiv: 2211.09110 [cs.CL].

Kwiatkowski, Tom et al. (2019). "Natural Questions: a Benchmark for Question Answering Research". In: *Transactions of the Association of Computational Linguistics*.

Huggingface (2023b). *Open LLM Leaderboard*. Accessed: 2023-06-10. URL: https://huggingface.co/spaces/HuggingFaceH4/open_llm_leaderboard.

Gao, Leo et al. (Sept. 2021). *A framework for few-shot language model evaluation*. Version v0.0.1. DOI: 10.5281/zenodo.5371628. URL: https://doi.org/10.5281/zenodo.5371628.

Chen, Hao et al. (2023). *Maybe Only 0.5% Data is Needed: A Preliminary Exploration of Low Training Data Instruction Tuning*. arXiv: 2305.09246 [cs.AI].

Kumar, Ananya et al. (2022). *Fine-Tuning can Distort Pretrained Features and Underperform Out-of-Distribution*. arXiv: 2202.10054 [cs.LG].

Brown, Tom B. et al. (2020). *Language Models are Few-Shot Learners*. arXiv: 2005.14165 [cs.CL].

Radford, Alec et al. (2018). "Language Models are Unsupervised Multitask Learners". In: URL: https://d4mucfpksywv.cloudfront.net/better-language-models/language-models.pdf.

Ouyang, Long et al. (2022). *Training language models to follow instructions with human feedback*. arXiv: 2203.02155 [cs.CL].

Schulman, John et al. (2017). *Proximal Policy Optimization Algorithms*. arXiv: 1707.06347 [cs.LG].

Hendrycks, Dan et al. (2021). *Measuring Massive Multitask Language Understanding*. arXiv: 2009.03300 [cs.CY].

Köpf, Andreas et al. (2023). *OpenAssistant Conversations – Democratizing Large Language Model Alignment*. arXiv: 2304.07327 [cs.CL].

Huggingface (2023a). *AutoTrain: Create powerful AI models without code*. URL: https://huggingface.co/autotrain.

Wei, Jason et al. (2022). *Emergent Abilities of Large Language Models*. arXiv: 2206.07682 [cs.CL].